

FINAL MINI PROJECT | COURSE CAPSTONE

Build a Python Application with a Local AI (RAG) Assistant

Nine project options, one shared requirement: your program must retrieve answers from your own local files and pass them to a language model running on your own machine.

What you build	What makes it an AI + RAG project
A complete, menu-driven Python application that uses classes, inheritance, modules, file persistence, exception handling and at least one algorithm you implement yourself (sorting, searching or recursion).	A separate assistant module that (1) retrieves the most relevant records from your own JSON / CSV / TXT files, (2) generates an answer with a local language model using only those records, and (3) cites the record IDs it used.

Zero cost and zero blockers - guaranteed

No API keys. No cloud accounts. No paid services. No database and no vector store. No third-party Python packages: everything in this handout uses only the Python standard library. You need an internet connection **once** to download the free Ollama runtime and a small model (about 1.4 GB). After that the project runs fully offline. If your computer cannot run a model at all, the required offline fallback mode keeps your project complete and gradeable.

Choose from	Or propose your own	Starter code	Model
9 ready-to-build project options in this handout, sections 5-13.	Any original problem statement that meets the same core-programming and AI + RAG expectations. Use the one-page proposal template in section 14.	A tested, standard-library starter kit is provided: <code>llm_client.py</code> , <code>rag.py</code> , <code>ai_assistant.py</code> .	qwen3:1.7b via Ollama, running on localhost. A smaller model is listed for low-memory machines.

Read sections 1 to 4 before you choose. They define the requirements, the grading rubric and the five tests every submission must pass. Sections 5 to 13 are the project options; each one is a single page and tells you exactly what to index, which field to use as the source ID, and what the assistant is not allowed to do.

SECTION 1

How this project works

Your application is the system of record. It owns the data, does the arithmetic and enforces the rules. The language model is a *presentation layer on top of retrieved facts*: it explains, summarises, drafts and recommends, but it never invents a record and never becomes the source of truth. That separation is what makes the project both accurate and gradeable.

The retrieve-then-generate pipeline

Retrieval-augmented generation (RAG) means the model only ever sees facts you selected for it. Your program answers the question "which of my records are relevant?" in ordinary Python; the model answers "how do I say this well?"

1 BUILD Finish and test the core Python application first. It must be useful with the AI feature switched off.	2 RETRIEVE Filter your local records with your existing project logic, then rank the survivors and keep the top 3-5.	3 GENERATE Send only those records plus the question to the local model, with instructions to use nothing else.	4 VERIFY Print the source IDs, test a question the files cannot answer, and stop the model to prove the app survives.
--	--	---	---

What travels where

```

User question                                Your local files (data/*.json, *.csv, *.txt)
|                                             |
|                                             | [1] load -> list of dicts / text chunks
|                                             |
+-----> [2] filter + rank (plain Python) --+
|
| v
| top 3-5 records, each with a stable source id
|
| v
| [3] prompt = rules + question + numbered sources
|
| v
| POST http://localhost:11434/api/generate (Ollama, on your machine)
|
| v
| [4] answer + "Sources used: R-001, R-003" -> printed with an AI label

```

The one rule that decides your grade

If the retrieved records do not contain the answer, the assistant must say so. An assistant that produces a confident answer from nothing is a failed AI + RAG feature, no matter how polished the rest of the program is.

SECTION 2

Requirements for every project

A. Core programming requirements

These apply to all nine options and to any proposal of your own.

1. **Classes and inheritance.** At least one base class and two child classes that genuinely specialise it (extra attributes and at least one overridden method).
2. **Modular structure.** Three or more focused modules plus a thin `main.py`. Keep the AI code in its own module (`ai_assistant.py` or `rag.py`) so the core application never depends on it.
3. **File persistence.** Data is saved to and reloaded from JSON, CSV or TXT. Restarting the program must not lose data.
4. **Exception handling.** Custom exception classes for your domain errors, plus specific handling of `FileNotFoundError`, `json.JSONDecodeError`, `ValueError` and `PermissionError`. No bare `except: blocks`.
5. **One algorithm implemented by hand.** Write your own merge sort, quick sort, binary search or recursive traversal, in your own function, with a short docstring stating its complexity. You may still use `sorted()` elsewhere in the program.
6. **Menu-driven interface** that validates every input and never crashes on bad typing.

B. AI + RAG requirements

1. The assistant accepts a **natural-language question** typed by the user.
2. Before generating, the program **retrieves** relevant local records: apply your project filters first, then rank and keep the top 3-5. Keyword ranking is sufficient; embeddings are an optional extension, not a requirement.
3. The prompt **instructs the model to use only the supplied sources** and to reply with a refusal sentinel when they are insufficient.
4. The output **displays the source IDs, names or file paths** that produced it, and is clearly labelled as AI generated.
5. **Graceful degradation is mandatory.** With Ollama stopped, the model missing, or the request timing out, the program prints a clear message, falls back to a deterministic offline summary of the retrieved records, and returns to the menu.
6. **No arithmetic by the model.** Totals, GPAs, reorder quantities, scaled ingredient amounts and durations are computed in Python and passed to the model as facts.

C. Data and safety

Use fictional or sample data only. Do not put real student records, real contacts, financial statements, medical information or anything you would not publish into your data folder or into a prompt. Small local models make mistakes: label every generated answer, and never present it as verified fact.

SECTION 3

Deliverables, milestones and grading

What you submit

#	Deliverable	Detail
1	Source code	All modules, runnable with <code>python main.py</code> . Standard library only.
2	data/ folder	Fictional sample data large enough for retrieval to be meaningful (about 15+ records, or 5+ text files).
3	README.md	Setup steps, the model tag you used, how to run, project structure, and three sample questions with their actual output including source IDs.
4	TESTS.md	The five required tests from section 4, each with the command, the expected behaviour and the observed output.
5	Demo	5-7 minutes: core features, one grounded answer, one refusal, and the app still working with Ollama stopped.

Suggested milestones

Stage	Focus	Done when
1	Design and data	Classes sketched, module list fixed, sample data files created and loading cleanly.
2	Core application	Menu, persistence, exceptions and your hand-written algorithm all working. No AI yet.
3	Retrieval	<code>retrieve()</code> returns sensible records for five test questions and nothing for an unrelated one.
4	Generation and hardening	Prompt grounded, source IDs printed, five required tests passing, README and demo ready.

Grading rubric (100 points)

Criterion	Pts	What full credit looks like
Object-oriented design	15	Base plus two child classes, meaningful overrides, no logic duplicated across classes.
Modular structure and style	10	Focused modules, docstrings, clear names, no dead code, AI code isolated.
File persistence	10	Save and load round-trip verified; corrupt or missing files handled, not crashed on.
Exceptions and validation	10	Custom exceptions, specific handlers, every menu input validated.
Hand-written algorithm	10	Correct implementation in your own code, tested on empty, single and duplicate input.
Retrieval quality	15	Filter then rank, top-k limited, stable source IDs attached, no match handled explicitly.
Grounded generation	10	Prompt forbids outside knowledge, refusal path works, output labelled as AI generated.
Resilience	10	Model stopped, model missing and timeout all handled; offline fallback produces a useful answer.
Documentation and demo	10	README, TESTS.md and demo match the code that was submitted.

Common ways to lose easy points

The AI module crashes the whole program when Ollama is off; the answer names records that were never retrieved; source IDs are missing; the model is asked to do the arithmetic; the data folder has three records so retrieval is trivial; sorting is claimed but `sorted()` is all that is used.

SECTION 4A | IMPLEMENTATION GUIDE

Guide 1: retrieve from your own local files

Your knowledge source is a folder of plain files that you create. Python loads them into lists of dictionaries or lists of text chunks, selects the most relevant items, and passes only those items to the model. No database, no vector store and no external package is required at any point.

Step 1 - choose the file format

Format	Best for	Standard-library tool	Source ID to use
JSON	Nested records: students, recipes, tasks, playlists, products with supplier blocks.	<code>json</code>	a dedicated id field you assign
CSV	Flat tabular records: grades, transactions, stock levels.	<code>csv.DictReader</code>	the row id column, or row number
TXT / MD	Notes, descriptions, reference documents, file contents.	<code>pathlib</code>	file name plus chunk number

Give **every** record a stable, human-readable ID before anything else: S-104, BK-0032, TXN-2211. Citations are only meaningful if the ID never changes when the file is re-sorted or re-saved.

Step 2 - load the files

```
from pathlib import Path
import csv, json

DATA = Path(__file__).parent / "data"

records = json.loads((DATA / "recipes.json").read_text(encoding="utf-8"))

with open(DATA / "grades.csv", newline="", encoding="utf-8") as handle:
    rows = list(csv.DictReader(handle))          # each row is a dict

notes = (DATA / "notes.txt").read_text(encoding="utf-8")
chunks = [{"id": f"notes.txt#{i}", "text": part.strip()}
           for i, part in enumerate(notes.split(chr(10) * 2), start=1)
           if part.strip()]                      # blank line = chunk boundary
```

Always pass `encoding="utf-8"`. Default encodings differ between Windows and macOS and are a frequent source of `UnicodeDecodeError` at demo time. Wrap every load in `try / except FileNotFoundError, json.JSONDecodeError` and start with an empty collection rather than crashing.

Step 3 - filter, then rank, then cap

Retrieval is two moves. First reuse the filters your application already has - availability, student ID, month, category, stock level, completion status. Then rank whatever survived by keyword overlap with the question and keep only the top few.

```
import json, re

STOPWORDS = {"the", "and", "for", "with", "what", "which", "how", "are", "from", "that",
             "this", "show", "list", "give", "find", "tell", "please", "about", "have"}

def terms(value):
    words = re.findall(r"[a-z0-9]+", str(value).lower())
    return {w for w in words if len(w) > 2 and w not in STOPWORDS}

def search_text(record):
    # Values only - never the field names. Otherwise a question containing
    # "minutes" matches every record that has a "minutes" field and ranking breaks.
    if isinstance(record, dict):
        return " ".join(json.dumps(v, default=str) for v in record.values())
    return json.dumps(record, default=str)

def retrieve(question, records, k=4):
    '''Return the k records with the most keyword overlap. O(n) per question.'''
    query = terms(question)
    scored = [(len(query & terms(search_text(r))), r) for r in records]
    scored = [pair for pair in scored if pair[0] > 0]
    scored.sort(key=lambda pair: pair[0], reverse=True)
    return [record for _score, record in scored[:k]]
```

Four retrieval mistakes to avoid

1. Scoring over the whole JSON string, field names included - every record ties. **2.** Returning everything: cap at 3-5 records or you will overflow the model context window and the model will silently ignore the tail. **3.** Forgetting the empty case: when nothing scores above zero, say so and never call the model. **4.** Sending huge text files whole: split TXT into paragraph-sized chunks and retrieve chunks, not files.

SECTION 4B | IMPLEMENTATION GUIDE

Guide 2: run a language model on your own machine

Ollama is a free runtime that downloads a model once and serves it over plain HTTP on `localhost`. Your Python code makes an ordinary POST request with `urllib` from the standard library, so there is no account, no key, no billing and nothing to install with `pip`.

Step 1 - one-time setup

1. Download and install Ollama for your operating system from <https://ollama.com/download>.
2. In Terminal or PowerShell run `ollama run qwen3:1.7b`. The first run downloads the model, so allow a few minutes and about 1.4 GB.
3. Ask it anything to confirm it replies, then type `/bye`. Leave the Ollama application running in the background.
4. Confirm the HTTP service is alive: open `http://localhost:11434` in a browser, or run `ollama list` to see the installed model tags.

Step 2 - pick a model that fits your machine

Model tag	Download	Comfortable on	Notes
<code>qwen3:1.7b</code>	~1.4 GB	8 GB RAM or more	Class default. It is a thinking model, so the starter code sends <code>"think": false</code> to suppress the reasoning trace and keep answers fast.
<code>gemma3:1b</code>	~815 MB	4-8 GB RAM	Lightest option and the fastest to respond. Use it if <code>qwen3</code> is slow or runs out of memory. Requires Ollama 0.6 or newer.
<code>llama3.2:1b</code>	~1.3 GB	4-8 GB RAM	Second lightweight alternative if you want to compare output quality.

Check the current requirements on the Ollama site before installing; model sizes are quantised downloads and change between releases. Expect a few seconds per answer on a laptop CPU. Larger models are not better for this project: the grade depends on retrieval and grounding, not on model size.

Step 3 - the client (standard library only)

```

# llm_client.py
import json
from urllib import error, request

HOST, MODEL, TIMEOUT = "http://localhost:11434", "qwen3:1.7b", 180

class LLMUnavailable(RuntimeError):
    '''Raised when the local model cannot produce a usable answer.'''

def is_available(timeout=3):
    try:
        with request.urlopen(f"{HOST}/api/tags", timeout=timeout) as response:
            return response.status == 200
    except OSError:
        return False

def _post(path, payload, timeout):
    req = request.Request(f"{HOST}{path}", data=json.dumps(payload).encode("utf-8"),
                          headers={"Content-Type": "application/json"})
    with request.urlopen(req, timeout=timeout) as response:
        return json.loads(response.read().decode("utf-8"))

def generate(prompt, model=MODEL, timeout=TIMEOUT):
    payload = {"model": model, "prompt": prompt, "stream": False,
              "think": False,           # hide the qwen3 reasoning trace
              "keep_alive": "10m",      # do not reload the model every question
              "options": {"temperature": 0.2, "num_ctx": 4096, "num_predict": 400}}

    try:
        try:
            body = _post("/api/generate", payload, timeout)
        except error.HTTPError as exc:
            if exc.code != 400:
                raise
            payload.pop("think")         # older Ollama builds reject this field
            body = _post("/api/generate", payload, timeout)
    except (OSError, ValueError) as exc:  # URLError, TimeoutError, invalid JSON
        raise LLMUnavailable(f"local model unavailable: {exc}") from exc
    text = str(body.get("response", "")).strip()
    if not text:
        raise LLMUnavailable("the model returned an empty response")
    return text

```

Why each option is there. stream: False returns one complete JSON object instead of a stream of fragments. temperature: 0.2 keeps answers close to the sources. num_predict caps the answer length so a small model cannot ramble for minutes. num_ctx must be large enough for your prompt: if your retrieved context is long, the overflow is silently discarded. Catching OSError covers URLError, HTTPError, connection refused and timeouts in a single, specific handler.

SECTION 4B CONTINUED

Guide 2: grounding the prompt

A grounded prompt has three parts: the rules, the question, and the numbered sources. The refusal sentinel gives your Python code something reliable to detect, instead of trying to interpret free-form hedging.

```
# rag.py (continued)
MAX_CONTEXT_CHARS = 4000

PROMPT_TEMPLATE = '''You answer questions for a Python class project.

Rules:
1. Use ONLY the numbered sources below. Never invent records, names or numbers.
2. If the sources do not contain the answer, reply exactly: NOT_IN_SOURCES
3. Finish with one line: "Sources used: <comma separated ids>"

Question: {question}

Sources:
{context}
'''

def build_context(records, id_field="id"):
    blocks, used = [], 0
    for record in records:
        source_id = record.get(id_field, "?")
        block = f"[Source {source_id}] {json.dumps(record, default=str)}"
        if used + len(block) > MAX_CONTEXT_CHARS:
            break
        blocks.append(block)
        used += len(block)
    return chr(10).join(blocks)

def offline_answer(question, records, id_field="id"):
    '''Deterministic fallback so the feature still works with no model.'''
    lines = ["[offline mode - local AI model not available]",
            f"Question: {question}",
            f"{len(records)} matching local record(s):"]
    lines += [f" - [{r.get(id_field, chr(63))}] {json.dumps(r, default=str)[:150]}"
              for r in records]
    return chr(10).join(lines)
```

The single entry point your menu calls

```
# ai_assistant.py
import llm_client, rag

def ask(question, records, id_field="id", k=4):
    '''Return (answer_text, retrieved_records). Never raises.'''
    if not question or not question.strip():
        return "Please enter a question.", []

    hits = rag.retrieve(question, records, k=k)          # 1. RETRIEVE
    if not hits:
        return "No matching records were found in the local files.", []

    prompt = rag.PROMPT_TEMPLATE.format(question=question.strip(),
                                         context=rag.build_context(hits, id_field))

    try:
        answer = llm_client.generate(prompt)            # 2. GENERATE
    except llm_client.LLMUnavailable as exc:
        print(f"[warning] {exc} - using offline summary")
        return rag.offline_answer(question, hits, id_field), hits

    if "NOT_IN_SOURCES" in answer:
        answer = "The local files do not contain enough information to answer that."

    ids = ", ".join(str(r.get(id_field, "?")) for r in hits) # 3. CITE
    return f"{answer}{chr(10)}{chr(10)}[AI generated - review before use] Sources: {ids}", hits
```

Design note

Notice that `ask()` never raises and always returns the records it used. Your menu code therefore needs no error handling of its own, and your demo can print the retrieved IDs next to the answer to prove the citation is real.

SECTION 4C | IMPLEMENTATION GUIDE

Guide 3: verify, then harden

The five required tests

Record all five in `TESTS.md` with the observed output. They are worth more marks than any extra feature you could add instead.

#	Test	How to run it	Required behaviour
1	Answerable question	Ask something your data clearly covers.	A useful answer, plus source IDs that really do contain the facts quoted.
2	Unanswerable question	Ask about something absent from your data, for example a record that does not exist.	Either no records retrieved, or the refusal message. No invented record, ever.
3	Model stopped	Quit the Ollama application, then use the assistant.	A clear warning, the offline fallback summary, and the menu still working.
4	Missing or corrupt data	Rename a data file; delete a closing brace in the JSON.	A specific message naming the file. No traceback.
5	Invalid input	Empty question; a menu choice such as abc or 99.	Re-prompt without crashing.

Troubleshooting

Symptom	Cause	Fix
URLError / connection refused	Ollama is not running.	Launch the Ollama app, or run <code>ollama serve</code> in a terminal.
HTTP 404, model not found	The tag was never pulled, or is misspelled.	<code>ollama pull qwen3:1.7b</code> , then check <code>ollama list</code> .
HTTP 400 mentioning an unknown field	An older Ollama build that predates the <code>think</code> option.	Update Ollama. The starter client already retries once without that field.
Answer starts with a long reasoning monologue	Thinking output is enabled.	Keep <code>"think": false</code> , or switch to <code>gemma3:1b</code> .
Very slow, or the first call times out	Model loading into memory on first use.	Warm it up once at start-up, set <code>keep_alive</code> , lower <code>num_predict</code> , use a smaller model.
Answer mentions records you did not retrieve	Prompt too loose, or too many sources sent.	Tighten the rules, drop <code>k</code> to 3, lower the temperature, and print the context you actually sent.
Long context appears ignored	The prompt exceeded the context window.	Cap <code>MAX_CONTEXT_CHARS</code> and raise <code>num_ctx</code> .
Out of memory / machine freezes	Model too large for available RAM.	Use <code>gemma3:1b</code> , close other applications, or demonstrate offline fallback mode.

Reference project layout

```
my_project/
  main.py          menu only, no business logic
  models.py        base class + child classes
  manager.py       collection logic, save / load
  algorithms.py    your own merge sort / binary search / recursion
  ai_assistant.py  ask(question, records) -> (answer, sources)
  rag.py           retrieve, build_context, offline_answer
  llm_client.py    the local HTTP call, LLMUnavailable
  data/
    records.json   fictional sample data with stable ids
    notes.txt      optional supporting text for retrieval
  README.md
  TESTS.md
```

Optional extensions (still free, still no keys)

Rank with TF-IDF instead of raw overlap. Add a synonym map so "cheap" also matches "budget". Use Ollama's local embedding endpoint with a small embedding model for semantic retrieval, then compute cosine similarity in pure Python. Keep multi-turn history with the chat endpoint. Show a side-by-side comparison of two local models. Attempt these only after all five required tests pass.

PROJECT 01 | SECTION 5

Student Grade Management System

Concepts covered: Classes, inheritance, exceptions, file I/O, sorting algorithms, retrieval, prompt grounding

Store, analyse and report on student records. The system supports different kinds of students, computes results reliably, persists everything to disk, sorts by several criteria, and then uses retrieval to turn one student's history into a study plan.

Core requirements

1. Base Student class with name, ID and a grades list; Undergraduate and Graduate children adding major or thesis topic and overriding the summary or GPA weighting.
2. Methods to add a grade, compute GPA and display a formatted record.
3. A GradeManager class holding many students, with add, remove, find and class-average operations.
4. Save and load in CSV or JSON; the file is the single source of truth on restart.
5. Raise custom exceptions for an out-of-range grade, a duplicate ID and an unknown student; handle file errors separately.
6. Implement merge sort or quick sort yourself to order students by GPA, name or ID.
7. Menu-driven interface with full input validation.

Required AI + RAG feature: AI Study Support Assistant

Design decision	Your specification
What to index	The selected student's record (courses, grades, dates) from your data file, plus a <code>topics.json</code> or <code>topics.txt</code> file in which each course or topic has a one-paragraph description and recommended practice.
Source ID field	<code>student_id</code> for records, <code>topic_code</code> for topic descriptions.
Example questions	"Which topics should S-104 review before the final exam?" / "Summarise S-102's strengths and weaknesses in one paragraph."
Python must compute	The GPA, every average, the pass or fail status, and the list of weakest courses. The model receives those numbers as facts.
The assistant must never	Invent a course, a grade or a student; recalculate a GPA; or comment on a student who was not retrieved.

Definition of done
Asking about a student with no weak topics produces an honest positive summary; asking about a student ID that does not exist produces a refusal, not an invented transcript; the printed GPA in the AI answer matches the GPA your Python code printed.

Optional stretch: Let the assistant compare a student against the anonymised class average that your code computes.

PROJECT 02 | SECTION 6

Library Book Management System

Concepts covered: Classes, inheritance, modules, file I/O, exceptions, searching algorithms, retrieval, grounding

Track materials, members and borrowing. Books, magazines and DVDs share a base class, and the catalogue becomes the knowledge source for a natural-language recommendation assistant that is physically incapable of recommending something the library does not own.

Core requirements

1. Modules such as `library_items.py`, `members.py`, `library_manager.py`, `search_utils.py`.
2. Base `LibraryItem` class with `Book`, `Magazine` and `DVD` children; each defines its own loan period.
3. A `Member` class with a borrowing history.
4. Check-out and check-in in a `LibraryManager`, including due-date calculation.
5. Persist catalogue and loans to file; reload at start-up.
6. Custom exceptions for item-not-found, already-borrowed and overdue.
7. Implement binary search yourself to find an item by ID - state in a comment that the list must be sorted by ID first, and sort it before searching.
8. Linear search by title or author substring; catalogue display sorted by several criteria.

Required AI + RAG feature: AI Catalogue Assistant

Design decision	Your specification
What to index	Catalogue records already filtered to <code>available == True</code> . Include a short synopsis, genre, page count or runtime in each record so ranking has something to match.
Source ID field	<code>item_id</code> , shown next to every recommended title.
Example questions	"Find a short mystery I can borrow today." / "What is a good follow-up for a member who enjoyed BK-0032?"
Python must compute	Availability, due dates, overdue status and the eligible candidate list.
The assistant must never	Recommend a title outside the retrieved candidates, or claim something is available when the filter said otherwise.

Definition of done
A request with no eligible match prints a clear "nothing available matches that" message and the model is not called at all; every recommendation shows an item ID that exists in your catalogue file.

Optional stretch: Add a reservation queue and let the assistant explain a member's position in it.

PROJECT 03 | SECTION 7

File Organiser and Document Search Tool

Concepts covered: Recursion, file I/O, modules, exceptions, searching, classes, text chunking, retrieval

Recursively scan a sandbox folder that you create inside the project, index the readable text files, and answer questions about their contents with file-path citations. This is the option closest to a real document-RAG system, so the safety limits below are part of the grade.

Core requirements

1. Modules such as `file_scanner.py`, `file_organizer.py`, `search_engine.py`.
2. A recursive directory walk that you write yourself, with depth reporting.
3. A `FileInfo` class for name, size, extension, absolute path and modified date.
4. Group files by extension in a dictionary and report statistics by type and total size.
5. Handle `PermissionError`, `FileNotFoundError`, `UnicodeDecodeError`, broken symbolic links and invalid paths - the scan continues and logs the skip.
6. Save the index to JSON so later searches do not rescan the disk.
7. Substring search by file name, plus a sorted index built with your own sorting algorithm.
8. **Hard limits:** operate only inside a configured `SANDBOX_DIR` inside the project, allow only `.txt` `.md` `.csv` `.py`, skip anything over about 200 KB, never move or delete a file without an explicit confirmation, and never follow a path that resolves outside the sandbox.

Required AI + RAG feature: AI Document Finder and Summariser

Design decision	Your specification
What to index	Text files inside the sandbox only, split into paragraph or 800-character chunks. Each chunk keeps its file path and chunk number.
Source ID field	<code>relative/path.txt#3</code> - path plus chunk number, printed for every claim.
Example questions	"Which of my notes mention recursion?" / "Summarise what these files say about exception handling."
Python must compute	The candidate chunk list, all counts and sizes, and the extension and size filtering.
The assistant must never	Cite a file that was not retrieved, or answer from general knowledge when the chunks do not cover the question.

Definition of done
Verify with `Path.resolve()` that no indexed path escapes the sandbox; a question about a topic absent from every file returns the refusal; a binary file dropped into the sandbox is skipped with a logged reason instead of crashing.

Optional stretch: Highlight the matching sentence inside each cited chunk before sending it to the model.

PROJECT 04 | SECTION 8

Contact Manager with Notes and Backup

Concepts covered: Classes, inheritance, file I/O, exceptions, sorting, searching, modules, retrieval, drafting

Manage fictional personal and business contacts with search, sorting and timestamped backups, then use the interaction notes as a knowledge source to summarise a relationship or draft a follow-up message.

Core requirements

1. Separate modules for the contact classes, file operations and search utilities.
2. Base Contact class with PersonalContact and BusinessContact children (birthday versus company and job title), each overriding the display method.
3. Validate email and phone formats with re and raise a custom ValidationError.
4. Detect duplicates on name plus email before adding.
5. Implement quick sort or merge sort yourself for name, date added or organisation; implement binary search for lookup on the sorted list.
6. Save to JSON, plus a timestamped backup file such as backup_20260415_1830.json, and recover cleanly from a corrupt primary file.
7. CSV import and export, with a per-row error report that skips bad rows and keeps going.
8. Filtered search combining several criteria.

Required AI + RAG feature: AI Contact Notes Assistant

Design decision	Your specification
What to index	Fictional interaction notes: one record per contact per meeting, each with a date and a short summary. Filter to the chosen contact before ranking.
Source ID field	contact_id plus note_id.
Example questions	"Summarise my relationship with C-007." / "Draft a short follow-up email to C-007 based on the last two meetings."
Python must compute	Which notes are the most recent, how many days since the last contact, and the contact filter.
The assistant must never	Invent a meeting, a commitment, a phone number or an email address; or emit a draft without the "AI generated draft" label.

Definition of done
All data is clearly fictional; drafts reference only retrieved note IDs; a contact with no notes produces "no interaction history on file" rather than a plausible fabrication.

Optional stretch: Detect contacts not touched for 90 days and let the assistant draft a re-engagement message.

PROJECT 05 | SECTION 9

Expense Tracker with Categories and Budgets

Concepts covered: Classes, inheritance, file I/O, exceptions, sorting, modules, aggregation, retrieval

Record income and expenses, group them by category, enforce budgets and produce monthly reports. The assistant then explains the numbers your code computed - it is explicitly forbidden from doing arithmetic, which makes this option the clearest demonstration of correct division of labour between Python and a language model.

Core requirements

1. Modules such as `transactions.py`, `categories.py`, `reports.py`, `file_handler.py`.
2. Base Transaction class with Income and Expense children that override how the amount affects a balance.
3. A Category class and a Budget class that tracks a monthly limit and reports the remaining amount.
4. Persist every transaction to JSON; reload on start-up with a running balance.
5. Validate amounts and dates: reject a negative expense, a non-numeric amount and an impossible date such as 2026-02-31, using `datetime.strptime` and a custom exception.
6. Implement your own sort for date, amount or category ordering.
7. Monthly report: total per category, budget usage, largest transactions, month-over-month change.
8. Search transactions by description, date range or amount range.

Required AI + RAG feature: AI Spending Insights Assistant

Design decision	Your specification
What to index	Two things: the transactions matching the question's month or category, and one Python-generated summary record holding the totals, budget usage and deltas.
Source ID field	<code>txn_id</code> for transactions, <code>SUMMARY-2026-03</code> for the computed summary.
Example questions	"Why was March food spending higher than February?" / "Where am I closest to breaking a budget?"
Python must compute	Every single number: totals, averages, percentages, differences and budget remainders. Add the phrase "do not perform any arithmetic; use the numbers as given" to your prompt rules.
The assistant must never	Produce a figure that is not in the retrieved records, give financial advice, or omit the informational-only label.

Definition of done
Every number in the AI explanation appears verbatim in the Python report printed just above it. Change one transaction and re-run: the explanation changes accordingly.

Optional stretch: Add a recurring-transaction detector and ask the assistant to comment on the fixed versus variable split that your code calculated.

PROJECT 06 | SECTION 10

Inventory Management with Suppliers

Concepts covered: Classes, inheritance, file I/O, exceptions, sorting, searching, modules, retrieval, prioritising

Run a small store's stock: products, suppliers, stock levels, expiry and reordering. Python decides what to reorder and how much; the assistant turns that into a prioritised, readable purchasing brief with supplier context.

Core requirements

1. Modules such as `products.py`, `suppliers.py`, `inventory.py`, `alerts.py`.
2. Base Product class with `PerishableProduct` (expiry date, days-to-expiry) and `ElectronicProduct` (warranty months) children, each overriding a `needs_attention()` rule.
3. A `Supplier` class with lead time and contact detail; products reference a supplier code.
4. An `Inventory` class for stock in, stock out and level queries.
5. Persist to file after each change, or auto-save on a counter; reload at start-up.
6. Custom exceptions for out-of-stock, unknown product code and unknown supplier.
7. Implement your own sort for name, price, stock level or expiry date, and a multi-criteria search.
8. Low-stock alerts and a reorder report: reorder quantity from an explicit, documented formula such as reorder level minus stock on hand, plus safety stock.

Required AI + RAG feature: AI Reorder Assistant

Design decision	Your specification
What to index	The products flagged by your low-stock and expiry rules, each joined to its supplier record, plus the computed reorder quantity.
Source ID field	<code>sku</code> and <code>supplier_id</code> .
Example questions	"What should I reorder first this week and from whom?" / "Which perishable items are at risk in the next seven days?"
Python must compute	Low-stock detection, days to expiry, reorder quantities and lead times.
The assistant must never	Change a quantity, invent a supplier or a price, or hide the fact that a product has no supplier on file - that case must be flagged explicitly.

Definition of done
Quantities in the brief are identical to the Python reorder report; a product whose supplier code is missing appears with an explicit "no supplier on file" warning rather than a plausible invented name.

Optional stretch: Group the brief by supplier so one purchase order covers several SKUs, with the totals computed in Python.

PROJECT 07 | SECTION 11

Recipe Book with Ingredient Search and Scaling

Concepts covered: Classes, file I/O, modules, searching, sorting, exceptions, unit handling, retrieval

A digital cookbook that searches by ingredient, scales quantities and organises recipes by category. The assistant recommends from what you actually have stored and names precisely which ingredients you are missing.

Core requirements

1. Modules such as `recipe.py`, `ingredient.py`, `cookbook.py`, `search.py`.
2. A `Recipe` class with an ingredient list, ordered steps, preparation time, servings and difficulty.
3. An `Ingredient` class with name, quantity and unit, validating against an allowed unit list.
4. A `Cookbook` class to add, remove, list and filter recipes; JSON persistence with a documented schema.
5. Custom exceptions for an unknown unit, a non-positive quantity and a duplicate recipe name.
6. Implement your own sort for preparation time, name or difficulty.
7. Search for recipes that contain all, or any, of a given set of ingredients.
8. Scaling: recompute every quantity for a new serving count in Python, rounding sensibly and leaving units unchanged.

Required AI + RAG feature: AI Recipe Assistant

Design decision	Your specification
What to index	Recipes filtered by any hard constraints in the question, such as a maximum preparation time, then ranked by ingredient overlap with what the user says they have.
Source ID field	<code>recipe_id</code> , printed with each suggestion.
Example questions	"I have chickpeas, rice and 20 minutes - what can I make?" / "What can I substitute for yogurt in R-003?"
Python must compute	Ingredient matching, missing-ingredient lists, preparation-time filtering and all scaled quantities.
The assistant must never	Invent a recipe or a step, restate scaled amounts from its own arithmetic, or suggest a substitution without labelling it as a suggestion to verify.

Definition of done
Suggestions only ever cite stored recipe IDs; the missing-ingredient list comes from your set comparison; an impossible request such as a two-minute roast returns a refusal instead of a fabricated recipe.

Optional stretch: Add a pantry file and generate a shopping list, computed in Python, for the chosen recipe.

PROJECT 08 | SECTION 12

Task Manager with Priorities and Daily Planning

Concepts covered: Classes, inheritance, file I/O, sorting, exceptions, modules, date handling, retrieval

Organise tasks by priority and due date, track completion and detect scheduling conflicts. The assistant then turns your computed shortlist into a realistic daily plan, using only tasks that already exist.

Core requirements

1. Modules such as `tasks.py`, `task_manager.py`, `file_ops.py`, `scheduling.py`.
2. Base Task class with `WorkTask`, `PersonalTask` and `RecurringTask` children; the recurring child overrides completion to schedule the next occurrence.
3. High, Medium and Low priority, with a documented ordering rule that combines priority and due date - and a stable tie-break.
4. A `TaskManager` class to add, complete, filter and report; persist with created and modified timestamps in ISO format.
5. Validate dates with `datetime` and raise a custom exception for an unparseable or past due date where that is not allowed.
6. Detect conflicts: two tasks with overlapping scheduled time windows on the same day.
7. Implement your own sort, or a heap-based priority queue, for the ordering rule.
8. Keyword search and an overdue-task report.

Required AI + RAG feature: AI Daily Planning Assistant

Design decision	Your specification
What to index	Incomplete tasks due within a window you choose, plus a Python-generated conflict list and the total estimated minutes.
Source ID field	<code>task_id</code> , cited beside every line of the plan.
Example questions	"Plan tomorrow and tell me what to start with." / "What is at risk of slipping this week?"
Python must compute	Which tasks are overdue, which windows overlap, the priority ordering and the time totals.
The assistant must never	Add a task that was not retrieved, move a due date, or claim there is no conflict when your conflict list says otherwise.

Definition of done
Every line of the generated plan maps to a real task ID; the plan respects the conflicts and due dates your code found; an empty task list produces "nothing is due" rather than an invented schedule.

Optional stretch: Ask for a second plan under a constraint such as four available hours, with the capacity check still done in Python.

PROJECT 09 | SECTION 13

Music Library Organiser and Playlist Builder

Concepts covered: Classes, inheritance, file I/O, modules, sorting, searching, exceptions, retrieval, curation

Manage a catalogue of track metadata, build playlists and generate statistics. The assistant curates a playlist for a mood, activity or duration - drawn strictly from the tracks you own. You work with metadata records only; no audio files are needed.

Core requirements

1. Modules such as `media.py`, `playlist.py`, `library.py`, `stats.py`.
2. Base `MediaItem` class (title, duration, play count) with `Song` (artist, album, genre, year) and `Podcast` or `AudioBook` (show, episode, narrator) children. Model albums and artists by composition, not inheritance - an album is not a kind of song.
3. A `Playlist` class that holds items, computes total duration and rejects duplicates.
4. A `MusicLibrary` class managing the whole collection with JSON persistence.
5. Validate durations as `mm:ss` or seconds; raise custom exceptions for a duplicate track and a malformed record.
6. Implement your own sort for title, artist, album, duration or play count.
7. Search by artist, album, genre or year, including partial matches.
8. Statistics: most played, longest, genre breakdown, total listening time.

Required AI + RAG feature: AI Playlist Curator

Design decision	Your specification
What to index	Tracks that pass the question's hard filters, such as genre or year, each with a mood or tag field so ranking has signal to work with.
Source ID field	<code>track_id</code> , listed for every track in the playlist.
Example questions	"Build a 30-minute focus playlist." / "Suggest an upbeat warm-up set from my 1990s rock tracks."
Python must compute	Filtering, the running total duration, the fit against the requested length, and duplicate prevention.
The assistant must never	Include a track that is not in the retrieved list, invent an artist or a release year, or state a total duration it calculated itself.

Definition of done
Every track in the saved playlist exists in the library file; the printed total duration comes from your Python sum; a request for a genre you do not own returns a refusal instead of invented songs.

Optional stretch: Ask the assistant to justify the ordering, then verify the transitions against the tempo or mood tags in your data.

SECTION 14

Propose your own project

An original idea is welcome and is graded against exactly the same rubric. It must be your own problem, use your own fictional local data, and satisfy every requirement in section 2. Get it approved before you start building.

What makes a good proposal

Test	Ask yourself
Local data exists	Can I write 15 or more fictional records, or 5 or more short text files, that a question could reasonably be answered from?
Retrieval is meaningful	Would a keyword query pick out a sensible subset? If every question needs all the data, there is nothing to retrieve.
Python owns the truth	Is there real computation - totals, ordering, matching, validation - that my code does and the model only describes?
Grounding is checkable	Can a grader look at a cited ID and confirm the claim is supported?
Scope is honest	Can the core application be finished before the AI feature is even started?

Proposal template - one page, submit for approval

1. Title and one-sentence problem statement
2. Core Python design
 - base class + two child classes (name the overridden methods)
 - modules and their responsibilities
 - the algorithm I will implement by hand, and where it is used
 - persistence format and file names
 - three custom exceptions and what triggers each
3. Local data
 - files, formats, and how many records or chunks
 - the field I will use as the source ID
 - how it is fictional and safe to share
4. AI + RAG feature
 - the feature name and what it produces
 - what I filter on, what I rank on, and my value of k
 - three example questions, one of which the data cannot answer
 - what Python computes and the model must not
 - what the assistant must never do
5. Risks and fallback
 - what I will demonstrate if my machine cannot run the model

Ideas that map cleanly onto these requirements

A course-syllabus question answerer over your own notes; a fitness or habit log with a weekly review assistant; a plant-care or maintenance scheduler; a board-game rules lookup over rule text you wrote; a travel-packing planner; a bug or ticket triage tool over fictional reports; a scholarship or grant matcher over fictional programme descriptions. Each one has records, filters, real computation and citable sources.

SECTION 15

Appendix: quick reference

Commands

Command	Purpose
<code>ollama run qwen3:1.7b</code>	Download (first time) and chat with the class model. Type <code>/bye</code> to exit.
<code>ollama pull gemma3:1b</code>	Download the lighter alternative model without starting a chat.
<code>ollama list</code>	Show the model tags installed on this machine.
<code>ollama ps</code>	Show which models are currently loaded in memory.
<code>ollama serve</code>	Start the local server manually if the desktop application is not running.
<code>ollama rm <tag></code>	Delete a model and reclaim the disk space.
<code>python demo_rag.py</code>	Run the starter-kit smoke test. Works with or without Ollama running.

Endpoints used by this project

Endpoint	Method	Used for
<code>http://localhost:11434/api/tags</code>	GET	Availability check, and confirming the model tag is installed.
<code>http://localhost:11434/api/generate</code>	POST	Single-turn generation. This is all the project requires.
<code>http://localhost:11434/api/chat</code>	POST	Optional: multi-turn conversations with message history.
<code>http://localhost:11434/api/embed</code>	POST	Optional extension: local embeddings for semantic retrieval.

Official documentation

- Install: <https://ollama.com/download>
- Model library and tags: <https://ollama.com/library>
- API reference: <https://docs.ollama.com> and <https://github.com/ollama/ollama/blob/main/docs/api.md>
- Python standard library: `json`, `csv`, `pathlib`, `urllib.request`, `re`, `datetime`, `unittest`

Academic honesty and AI use

You are expected to use a language model *inside* your program - that is the point of the assignment. If you also use an AI tool to help write your code, say so in your README: name the tool, describe what it helped with, and be ready to explain any line of your submission line by line during the demo. Code you cannot explain will not receive credit. Submitting another student's project, or a downloaded project, as your own is plagiarism.

Final pre-submission checklist

	Check
1	<code>python main.py</code> runs on a clean copy of the folder, with no missing imports.
2	No third-party packages, no API keys, no database, no hard-coded absolute paths from your own computer.
3	Base class plus two children, and the hand-written algorithm, are both easy to point to.

	Check
4	Data is fictional and the data folder is included in the submission.
5	The assistant prints source IDs on every answer, and an AI-generated label.
6	A question the data cannot answer is refused, not answered.
7	With Ollama stopped, the program warns, falls back, and keeps running.
8	README and TESTS.md are complete, with real captured output.

Build the program first. Retrieve carefully. Let the model explain, never decide. Cite everything.